



Flight Dynamics and Control of Aerospace Vehicle

Python Project : AirCraft Simulation

Name: JANNTALLAH SABRY ABASS

Prof.: Ahmed Aboulftouh

This code simulates the dynamics of a six-degree-of-freedom (6-DoF) aircraft model, specifically an F-16, using differential equations to represent its motion. Here is a breakdown of the key components and equations in the code:

- Equations

Evaluate Equations at t_i

Translational Equations

$$\begin{aligned}\dot{u} &= \frac{1}{m} F_{x_b} - G s_\theta - wq + vr \\ \dot{v} &= \frac{1}{m} F_{y_b} + G s_\phi c_\theta - ur + wp \\ \dot{w} &= \frac{1}{m} F_{z_b} + G c_\phi c_\theta - vp + uq\end{aligned}$$

Rotational Equations

$$\begin{aligned}\dot{p} &= \frac{J_{xz}(J_x - J_y + J_z)pq - [J_z(J_z - J_y) + J_{xz}^2]qr - J_zL + J_{xz}N}{J_x J_z - J_{xz}^2} \\ \dot{q} &= (J_z - J_x)rp - J_{xz}(p^2 - r^2) + M \\ \dot{r} &= \frac{-J_{xz}(J_x - J_y + J_z)qr - [J_x(J_x - J_y) + J_{xz}^2]pq - J_{xz}L + J_xN}{J_x J_z - J_{xz}^2}\end{aligned}$$

Euler Kinematic Equations

$$\begin{bmatrix} \dot{\phi} \\ \dot{\theta} \\ \dot{\psi} \end{bmatrix} = \begin{bmatrix} 1 & \sin(\phi) \tan(\theta) & \cos(\phi) \tan(\theta) \\ 0 & \cos(\phi) & -\sin(\phi) \\ 0 & \sin(\phi) \sec(\theta) & \cos(\phi) \sec(\theta) \end{bmatrix} \begin{bmatrix} p \\ q \\ r \end{bmatrix}$$

Components and Explanation

1. State Vector (x):
 - x[0] = u_b_mps: Axial velocity in the body-fixed coordinate system.
 - x[1] = v_b_mps: Sideways velocity in the body-fixed coordinate system.
 - x[2] = w_b_mps: Vertical velocity in the body-fixed coordinate system.
 - x[3] = p_b_rps: Roll angular velocity.
 - x[4] = q_b_rps: Pitch angular velocity.
 - x[5] = r_b_rps: Yaw angular velocity.
 - x[6] = phi_rad: Roll angle.
 - x[7] = theta_rad: Pitch angle.

- $x[8] = \text{psi_rad}$: Yaw angle.

2. Constants:

- Mass (m_{kg}): 15000 kg.
- Moments of inertia ($J_{xx_b_kgm2}$, $J_{yy_b_kgm2}$, $J_{zz_b_kgm2}$, $J_{xz_b_kgm2}$): Represent resistance to angular acceleration about the respective axes.
k

3. Gravitational Forces:

- $g_{z_n_mps2}$: Gravitational acceleration (9.81 m/s²).
- Components of gravitational acceleration resolved in the body coordinate system ($g_{x_b_mps2}$, $g_{y_b_mps2}$, $g_{z_b_mps2}$).

4. Aerodynamic Forces and Moments:

- Lift (L) and Drag (D): Calculated based on airspeed and angle of attack (α).
- Thrust (T) and its angular component (T_{theta}).
- External forces ($F_{x_b_kgmps2}$, $F_{y_b_kgmps2}$, $F_{z_b_kgmps2}$): Resulting forces in the body coordinate system.
- External moments ($l_{_b_kgmps2}$, $m_{_b_kgmps2}$, $n_{_b_kgmps2}$): Resulting torques.

5. Equations of Motion:

- Translational Motion:

$u_{\text{dot_b}}$, $v_{\text{dot_b}}$, $w_{\text{dot_b}}$: Acceleration components in the body-fixed coordinate system.

- Rotational Motion:

$p_{\text{dot_b}}$, $q_{\text{dot_b}}$, $r_{\text{dot_b}}$: Angular acceleration components.

6. Kinematics:

- ϕ_{dot} , θ_{dot} , ψ_{dot} : Rates of change of roll, pitch, and yaw angles.

7. Integration:

- The `solve_ivp` function from the `scipy.integrate` module integrates these equations over time using the Runge-Kutta method (RK45).

Detailed Code Breakdown

- Aircraft Model Data : Specifies the mass and moments of inertia of the aircraft.
- Extracts state variables from the input state vector x.

```
import numpy as np
from scipy.integrate import odeint
def rates(t,x):
    # Aircraft model data
    m_kg = 15000 # Mass of the F16 aircraft in kg
    Jxz_b_kgm2 = 0 # Moment of inertia in xz direction in kg*m^2
    Jxx_b_kgm2 = 50000 # Moment of inertia in xx direction in kg*m^2
    Jyy_b_kgm2 = 50000 # Moment of inertia in yy direction in kg*m^2
    Jzz_b_kgm2 = 70000 # Moment of inertia in zz direction in kg*m^2

    # state vectors
    u_b_mps = x[0]
    v_b_mps = x[1]
    w_b_mps = x[2]
    p_b_rps = x[3]
    q_b_rps = x[4]
    r_b_rps = x[5]
    phi_rad = x[6]
    theta_rad = x[7]
    psi_rad = x[8]
    #p1_n_m = x[9]
    #p2_n_m = x[10]
    #p3_n_m = x[11]

    #u_e = np.cos(psi_rad)*np.cos(theta_rad)*u_b_mps + (np.cos(psi_rad)*np.sin(theta_rad)*np.sin(phi_rad)-np.sin(psi_rad)*np.cos(phi_rad))*v_b_mps + (np.cos(psi_rad)*np.sin(theta_rad)*np.sin(phi_rad)+np.sin(psi_rad)*np.cos(phi_rad))*w_b_mps
    #v_e = np.sin(psi_rad)*np.cos(theta_rad)*u_b_mps + (np.sin(psi_rad)*np.sin(theta_rad)*np.sin(phi_rad)+np.cos(psi_rad)*np.cos(phi_rad))*v_b_mps + (np.sin(psi_rad)*np.sin(theta_rad)*np.sin(phi_rad)-np.cos(psi_rad)*np.cos(phi_rad))*w_b_mps
    #w_e = -np.sin(theta_rad)*u_b_mps + np.cos(theta_rad)*np.cos(phi_rad)*v_b_mps + np.cos(theta_rad)*np.sin(phi_rad)*w_b_mps

    #t = np.linspace(0, 10, 100)
    #result = odeint([u_e,v_e,z_e], [p1_n_m,p2_n_m,p3_n_m], t)
    # Extract the positions from the result
    #x_e = result[:, 3]
    #y_e = result[:, 4]
    #z_e = result[:, 5]
```

- Translational Equation
- Resolves gravitational force components in the body-fixed coordinate system
- Calculates lift, drag, and thrust forces.
- Computes the forces acting on the aircraft in the body-fixed coordinate system.
- Defines the translational acceleration components.

$$\begin{aligned}
m(\dot{U} + QW - RV) &= F_{G_x} + F_{A_x} + F_{T_x} \\
m(\dot{V} + RU - PW) &= F_{G_y} + F_{A_y} + F_{T_y} \\
m(\dot{W} + PV - QU) &= F_{G_z} + F_{A_z} + F_{T_z}
\end{aligned}$$

$$\begin{aligned}
m(\dot{U} + QW - RV) &= -mg \sin \Theta + (-D \cos A + L \sin A) + T \cos \Phi_T \\
m(\dot{V} + RU - PW) &= mg \sin \Phi \cos \Theta + F_{A_y} + F_{T_y} \\
m(\dot{W} + PV - QU) &= mg \cos \Phi \cos \Theta + (-D \sin A - L \cos A) - T \sin \Phi_T
\end{aligned}$$

```

# Gravity acts normal to Earth tangent CS
gz_n_mps2 = 9.81      # weight force N in z direction
L = -gz_n_mps2        # lift force in - z direction
drag_coe = 0.02
airspeed = np.linalg.norm(u_b_mps, v_b_mps, w_b_mps)
D = 0.5*drag_coe * airspeed**2
alpha = np.radians(np.arctan2(w_b_mps/u_b_mps))
T = 6071.43
T_theta = 0
# resolve gravity in body coor sys
gx_b_mps2 = -np.sin(theta_rad) * gz_n_mps2
gy_b_mps2 = np.sin(phi_rad) * np.cos(theta_rad) * gz_n_mps2
gz_b_mps2 = np.cos(phi_rad) * np.cos(theta_rad) * gz_n_mps2

# External forces
Fx_b_kgmps2 = (-gz_n_mps2*np.sin(theta_rad)) + (-D*np.cos(alpha)+L*np.sin(alpha)) + (T*np.cos()) # = FGx+FAx+FTx
Fy_b_kgmps2 = (gz_n_mps2*np.cos(theta_rad)*np.sin(phi_rad)) + () + ()
Fz_b_kgmps2 = (gz_n_mps2*np.cos(phi_rad)*np.cos(theta_rad)) + (-D*np.sin(alpha)-L*np.cos(alpha)) - (T*np.sin(T_theta))

# External Moments(torque)
l_b_kgmps2 = 0
m_b_kgmps2 = 0
n_b_kgmps2 = 0

# denominator in roll and yaw rate equation
Den = Jxx_b_kgm2 * Jzz_b_kgm2 - Jxz_b_kgm2**2

# translational equation
# x-axis (roll-axis) velocity equation , state u_b_mps
u_dot_b = (Fx_b_kgmps2/m_kg) + gx_b_mps2 - (w_b_mps * q_b_rps) + (v_b_mps * r_b_rps) # u_dot (acceleration)
# y-axis (pitch-axis) velocity equation , state v_b_mps
v_dot_b = (Fy_b_kgmps2/m_kg) + gy_b_mps2 - (u_b_mps * r_b_rps) + (w_b_mps * p_b_rps) # v_dot
# x-axis (roll-axis) velocity equation , state u_b_mps
w_dot_b = (Fz_b_kgmps2/m_kg) + gz_b_mps2 - (v_b_mps * p_b_rps) + (u_b_mps * q_b_rps) # w_dot

```

- Rotational Equation:
- Defines the rotational acceleration components.

$$\begin{aligned}
\dot{P}I_{xx} + QR(I_{zz} - I_{yy}) - (\dot{R} + PQ)I_{xz} &= L \\
\dot{Q}I_{yy} - PR(I_{zz} - I_{xx}) + (P^2 - R^2)I_{xz} &= M \\
\dot{R}I_{zz} + PQ(I_{yy} - I_{xx}) + (QR - \dot{P})I_{xz} &= N
\end{aligned}$$

```

#rotational equation
# roll equation
p_dot_b = (Jxz_b_kgm2*(Jxx_b_kgm2 - Jyy_b_kgm2 + Jzz_b_kgm2) * p_b_rps * q_b_rps - (Jzz_b_kgm2 * (Jzz_b_kgm2 - Jyy_b_kgm2) + Jxz_b_kgm2**2) * q_b_rps * r_b_rps - Jzz_b_kgm2 * l_b_kgmps2) / Den
# pitch equation
q_dot_b = (Jzz_b_kgm2 - Jxx_b_kgm2) * p_b_rps * r_b_rps - Jxz_b_kgm2 * (p_b_rps**2 - r_b_rps**2) + m_b_kgmps2
# yaw equation
r_dot_b = (-Jxz_b_kgm2*(Jxx_b_kgm2 - Jyy_b_kgm2 + Jzz_b_kgm2)*q_b_rps*r_b_rps - (Jxx_b_kgm2*(Jxx_b_kgm2 - Jyy_b_kgm2) + Jxz_b_kgm2**2)*p_b_rps*q_b_rps - Jxz_b_kgm2*l_b_kgmps2 + Jxx_b_kgm2 * n_b_kgmps2) / Den

```

- Kinematics Equation:
- Defines the rates of change of the orientation angles (roll, pitch, yaw).
- From Earth coordinate to body coordinate

$$\begin{bmatrix} \dot{\phi} \\ \dot{\theta} \\ \dot{\psi} \end{bmatrix} = \begin{bmatrix} 1 & \tan \theta \sin \phi & \tan \theta \cos \phi \\ 0 & \cos \phi & -\sin \phi \\ 0 & \sec \theta \sin \phi & \sec \theta \cos \phi \end{bmatrix} \begin{bmatrix} p \\ q \\ r \end{bmatrix}$$

```
# kinematics equation
phi_dot = p_b_rps + (np.sin(phi_rad)*np.tan(theta_rad)*q_b_rps) + (np.cos(phi_rad)* np.tan(theta_rad) *r_b_rps)
theta_dot = (np.cos(phi_rad) * q_b_rps) - (np.sin(phi_rad)*r_b_rps)
psi_dot = (np.sin(phi_rad) / np.cos(theta_rad)* q_b_rps) + (np.cos(phi_rad) / np.cos(theta_rad) * r_b_rps )

# position (navigation) equation
return [u_dot_b,v_dot_b,w_dot_b, p_dot_b , q_dot_b , r_dot_b , phi_dot,theta_dot,psi_dot ]
```

- Integration:
- Integrates the equations of motion over time (from 1 to 100 seconds) using the Runge-Kutta method and prints the solution.
- Initial conditions
- the body rates are related to Euler rates through the relation

```
from scipy.integrate import solve_ivp

# Initial conditions for F16 aircraft
u_b_mps = 0 # Axial velocity in m/s
v_b_mps = 0 # Sideways velocity in m/s (assuming level flight)
w_b_mps = 0 # Vertical velocity in m/s (assuming level flight)
p_b_rps = 0 # Roll rate in radians per second
q_b_rps = 0 # Pitch rate in radians per second
r_b_rps = 0 # Yaw rate in radians per second
phi_rad = 0 # Roll angle in radians
theta_rad = 0 # Pitch angle in radians
psi_rad = 0 # Yaw angle in radians

#p1_n_m = 0 # Initial x-axis position in meters
#p2_n_m = 0 # Initial y-axis position in meters
#p3_n_m = 0 # Initial z-axis position in meters

# Concatenate initial conditions and aircraft model data
initial_conditions = [u_b_mps, v_b_mps, w_b_mps, p_b_rps, q_b_rps, r_b_rps, phi_rad, theta_rad, psi_rad]

sol = solve_ivp(rates , [1,100] , initial_conditions , method='RK45')
print(sol)
```

```
message: The solver successfully reached the end of the integration interval.
success: True
status: 0
t: [ 1.000e+00  1.000e+00  1.001e+00  1.011e+00  1.111e+00
     2.111e+00  1.211e+01  1.000e+02]
y: [[ 0.000e+00  3.333e-05 ...  3.704e+00  3.300e+01]
     [ 0.000e+00  1.333e-05 ...  1.481e+00  1.320e+01]
     ...
     [ 0.000e+00  0.000e+00 ...  0.000e+00  0.000e+00]
     [ 0.000e+00  0.000e+00 ...  0.000e+00  0.000e+00]]
sol: None
```

Pytho

To determine Angles pitch,yaw,roll using inverse kinematics

1. Initial Data:

- Accelerations (a_x , a_y , a_z) from accelerometers.
- Angular rates ($\dot{p}$, $\dot{q}$, $\dot{r}$) from gyroscopes.
- Initial conditions of position, velocity, and attitude.

2. Equations of Motion:

- Translational motion equations (Newton's second law)
- Rotational motion equations (Euler's equations)

3. Integration:

- Integrate angular rates to get angular positions
- Integrate accelerations to get velocities and positions, considering initial velocity and position

By continuously updating these calculations with real-time sensor data, the aircraft's flight dynamics and control can be accurately managed and monitored. Advanced algorithms and onboard computers ensure that these integrations and calculations are performed quickly and accurately, allowing for precise control and stability of the aircraft.

- Summary

This code models the dynamics of an aircraft, considering both translational and rotational motion. The dynamics are governed by Newton's second law for translation and Euler's equations for rotation. The `solve_ivp` function is used to numerically integrate these equations, providing the state of the aircraft over time.